

Lelañea Product Description

Version 0.3, working draft. 21 August 2026. Built on the Daybreak framework. Ships alongside six authored JSON content files, the starter design system, and the approved design prototype. Every companion file is listed in the appendix.

Release 1 carries no module content. Not even Values. The first release ships the container: the public site, the shell in both themes, accounts and consent, onboarding, and the conversation itself. The modules appear as a real, navigable map with empty interiors, and Values arrives in release 2. See §10.

Status of this document. This describes what Lelañea is, what it does for a user, and what has to exist for that to be true. It is the input to a phased implementation plan, not the plan itself. Material drawn from the grounding brief and the framework overview is stated as fact. Anything proposed rather than sourced is marked *Proposed*, so it can be accepted, changed, or cut. Section 14 lists every open decision.

1. Product summary

Lelañea is a transcendental-coaching companion app built around the work of **Lelañea Fulton**: Professional Certified Coach (PCC, ICF), Transcendental Coach, and Unity-Consciousness Guide. It takes a body of coaching work that has historically only been available through 1:1 sessions, retreats, and private mentorship, and makes it available to anyone, at their own pace, regardless of financial means.

The design goal is that it feels less like software and more like having a coach beside you.

What it is not: not therapy, not healthcare, not crisis support. This is a hard legal and ethical line rather than a disclaimer to be buried. It shapes how agents are prompted, what they refuse, and what the user must acknowledge before they can begin.

The philosophy, in her terms:

- **Remembrance rather than self-improvement.** The work is not becoming someone new. It is remembering who the user has always been beneath conditioning, inherited belief, and accumulated identity.
- **The lowercase self and the capital Self.** The conditioned personality (ego) as distinct from the deeper awareness contemplative traditions call the Higher Self.
- **Purpose is internal first.** It begins in the user's relationship with themselves, not in an external destination.
- **Sovereignty over instruction.** The work is to strengthen someone's own discernment, not to substitute for it. Her writing is explicit that users retain their own judgment and take only what resonates. Everything the app recommends, offers, or concludes is held to that.
- **Lineage, not originality.** The influences are cited openly: Jung, Schwartz, Porges, Levine, van der Kolk, Kabat-Zinn, Kornfield, Brach, Maslow, Assagioli, Frankl, and the Vedanta, Buddhist, Taoist, and mystical traditions. The later modules move into Lelañea's own frameworks.

Voice register: warm, direct, unhurried, philosophically literate, comfortable holding paradox. Capturing this as a working artifact the models are prompted with, rather than as a list of adjectives, is treated as its own piece of the product (§5). Nothing is framed as good or bad, success or failure. Discomfort is treated as information rather than as evidence that something has gone wrong. Where the source copy uses a single-sentence-per-line cadence, that is deliberate and must not be reflowed into prose.

2. Target users

Proposed. The source material implies this audience but never states it directly.

Someone who has done well by external measures and finds the internal picture does not match. Someone in a threshold moment: a separation, a job loss, a bereavement, a decision that will not resolve. Someone already reading in this space who wants structure rather than more content. The common thread is a willingness to sit with a question rather than wanting a fix, and the app should filter gently for that rather than trying to convert everyone.

Not for: anyone in acute crisis, anyone seeking diagnosis or treatment, anyone wanting productivity coaching. The app should recognize these and redirect rather than serve them badly.

3. User experience

3.1 Public site and waitlist

The public site is the first contact, and for most visitors the only one. Its job is to explain the work honestly enough that the right people join and the wrong people do not.

The prototype (§6.1) contains the built version of this page, and its copy is real and considered rather than placeholder. Reuse it as written: the hero and lotus mark, the waitlist form, “what this is and what this is not”, the tiers, the quote bands, and the footer.

An honest statement of why this is being built. In her words, drawn from the mission and about documents already written: coaching of this kind has been reserved for people who can afford it, and the intent is to make it available regardless of financial means. What it is and what it is not (§1) belongs here too, plainly, rather than in small type at the bottom. Someone looking for therapy should be able to work that out before they sign up.

A video of Lelañea describing the app in her own words. Text explains the product; seeing and hearing her is what tells a visitor whether this is a person they want to spend time with. This is a **todo for Lelañea** (see §14). Transcribed, it also becomes early material for the voice fingerprint.

Waitlist capture works in two modes, switched by configuration.

- **Form mode.** Email address, name, where they heard about the app, and what they would want to achieve through using it. Fast, cheap, works without a model in the loop, and has almost no abuse surface.
- **Conversational mode.** A short exchange in her voice that gathers the same things by asking rather than by field. It doubles as a teaser: the visitor gets a genuine taste of what the app is actually like, which no landing page copy can convey.

The mode is a switch, not a fork. Both modes write the same fields to the same place, so the choice is a configuration decision rather than two implementations. Whichever is used, the answers carry provenance and become the seed of that person's profile if they later sign up, so what someone said they wanted before signing up is not thrown away at the point they do.

Cautions for the conversational mode. It is unauthenticated and costs tokens per visitor, so it needs rate limiting, abuse protection, and a hard cost ceiling. It is also the app's first impression, so it must run on the fingerprint rather than on a generic assistant voice. And it is a taste rather than a session: it should not slip into coaching, and the crisis redirect applies before there is any account at all (§12).

Having both modes also makes a real question cheap to answer: which converts better, and at what cost per signup.

3.2 How users arrive

On any given visit, a user is doing one of three things:

1. **Continuing the journey**, working through the next module in sequence.
2. **Bringing a live situation.** "I'm stuck on a decision; what do my values say?" The app answers through the work already done.
3. **Talking ad hoc**, thinking out loud about something on their mind.

All three are served from the same entry point. Navigation is chat-driven and only partly linear.

3.3 Screen layout and the chat interface

Chat is the primary interface. Everything else is arranged around it.

On desktop the shell has four columns, as built in the prototype (§6.1):

1. **Left nav.** The conversation, the workspace, then the journey, life situations, and share with Lelañea, then usage and billing and settings, with the account at the foot. Collapsible to icons, and a drawer below 900px.
2. **The conversation.** Collapsible, so someone can give the workspace the whole screen.
3. **The workspace.** Where modules, the journey, situations, resources, billing, settings, and account are worked with, and where the app's understanding becomes visible: data-slots forming, the profile assembling, a situation taking shape.
4. **The right rail.** Buttons that open drawers over the content rather than displacing it: the map of the sixteen modules, and resources.

Below 1240px the panes take turns rather than sitting side by side, and below 900px the nav becomes a drawer.

The core design idea is the pairing of the middle two columns. One is where someone speaks; the other is where they see the consequence of having spoken and can interact with it. It should provoke curiosity about what has been recorded and why, because a user experimenting with their own picture is engaged in the work rather than in an interface.

The chat input is unmistakably the main control: a box several rows tall, with **send and microphone buttons** in its footer. It should invite more than a single sentence.

While the model works, a plain **thinking** indicator shows that something is happening. The reply then **streams**, arriving as if typed rather than in a block. Pacing matters more here than in most products, because a full reply arriving at once reads as a lecture.

Under every response sits a timestamp and a one-line account of what that turn did. A chevron expands the detail, written in plain English rather than system language:

Updated what I understand about your relationship with your brother. Opened the Values module at the alignment audit. This turn used about 4,000 tokens.

Anything the turn touched is disclosed there: a slot written, a module instructed, a capability invoked, a situation amended, the cost incurred. The running cost total lives in the budget view (§3.20); the per-turn figure sits here, next to the thing that spent it.

The chat also acts as a command surface. A user can say what they want in plain language: open the boundaries module, record how I'm feeling today, log something going on with my sister. Some of these hand off to a specialist agent. Telling the app about a life event or a difficult situation brings in the agent that knows what to ask about a situation and how to ask it (§3.13).

The target feel is a 1:1 session with a coach who has your history in mind, rather than a support chat. It speaks in her voice through the fingerprint described in §5, which is what separates this from a chat window with a nice font.

3.4 Voice preferences

The real Lelañea does not speak to every client the same way, and neither should the app. Some people need it philosophical; others need the same point landed plainly and briefly.

The user can set their own **leanings**, adjusting the register along dimensions such as:

- **Philosophical** ↔ **grounded and practical**
- **Spiritual and devotional** ↔ **secular and plain**
- **Gentle** ↔ **direct, and further, challenging**
- **Encouraging** ↔ **neutral and unsentimental**
- **Verbose and exploratory** ↔ **concise and spare**
- **Empathetic and warm** ↔ **cool and analytical**
- **Energetic** ↔ **slow and spacious**
- **Question-led** ↔ **guidance-led**
- **Story and metaphor** ↔ **literal**
- **Playful** ↔ **serious**
- **Formal** ↔ **familiar**

Set once, adjustable at any time, and adjustable **mid-conversation**. "That was too much, be plainer with me" should work as a sentence rather than requiring a settings screen.

The essential fingerprint never dissolves. These are filters over her voice, not replacements for it. Her cadence, her ethics, what she will not say, how she grounds a claim, her refusal to frame things as success and failure: all invariant. No combination of settings produces a generic assistant wearing her name. This mirrors what the real Lelañea does when she adjusts her tone for the person in front of her; the manner moves, the person underneath does not.

Two registers sit above these preferences and are chosen by the app rather than the user: **guiding and teaching** (§3.14). The user sets the leanings; the moment sets the register.

3.5 Module navigation

The sixteen modules are ordered, and the order carries meaning: foundations before inner authority, inner authority before embodied relationship, and only then integration and expansion. That is **a recommended spine rather than a track**. `lelanea_module_structure.json` holds the sequence; nothing in it obliges a user to walk it in line.

The UI gives a **sense of place** (where you are, where you have been, what sits next to you) while making it plain that the whole map is open. The intended feeling is possibility rather than a corridor.

The prototype settles the form: **a map drawer**, opened from the right rail, showing the sixteen modules with what is done and what is current. What matters sits underneath that choice:

- The whole journey is always visible, including the parts not yet touched.
- **Jumping is a first-class action rather than an escape hatch.** A user who wants Boundaries on their second day should be able to go there without feeling they have broken something.
- Some modules are highlighted as more apt right now, for reasons the user can see and ignore (§3.7).
- The app never scolds anyone for leaving a module unfinished, because a module is never finished.

3.6 Coverage: breadth and depth

A module is never complete. Someone works through Values, and three years later a separation or a bereavement makes it a different module. Returning is not repetition; it is the point.

So there are no completion percentages, no ticks, no “3 of 12 steps”. The app shows **breadth and depth** instead: how much of a module’s territory has been touched, and how far in the user went, including how long they lingered, how much they wrote, how often they have returned, and what they left behind.

Breadcrumbs record where time and attention have gone. They can also prompt gently: a part of a module never opened, a value marked yellow eighteen months ago and never revisited, a reflection answered in a single line when everything around it ran long. An invitation to deepen, phrased as an invitation.

This lives in a panel or drawer **the user controls**: expanded, collapsed, or hidden entirely. Someone who finds it orienting keeps it open; someone who finds it a pressure closes it and the app does not argue. That control is a statement about whose journey this is.

The constraint that follows: breadth and depth are descriptions, never scores. Not a number, not a grade, not comparative, and never framed as distance from an ideal. If coverage becomes something to maximize, the app has become the thing it was built against.

3.7 Recommendation: what serves the user

Underneath both the map and the conversation sits one question: **what is genuinely in this person’s interest right now?** Not what would keep them in the app, and not what they last clicked.

It draws on everything the app knows: personality and disposition, goals and aspirations, struggles, relationships, work, spiritual development, current life situation and its constraints. It is **weighted toward the recent**, so what is live in this week’s conversations counts for more than what was

true a year ago. Someone who has spent three sessions circling a decision they cannot make is signalling something, and the app should notice.

It never instructs. Options are offered with the reason attached (“this has come up in the last few things you’ve written”) and the choice stays with the user. They can decline and not be asked again. They can ask why something is being suggested and get an honest answer from the underlying data rather than a rationalization.

That restraint is the product working as intended rather than politeness. The aim is the user’s **agency and sovereignty**: the capacity to weigh options and make choices driven by their own values and standards. Lelañea’s writing is explicit that users retain their own discernment and take only what resonates, so a recommender that quietly overrode that would contradict the work it was built to deliver.

The mechanism, deterministic rules bounding a space that agents then evaluate semantically, is described in §5.

3.8 Session recap on return

On return, the app opens as a coach opens a session: it surfaces what has changed since last time and asks what has shifted. It does not re-ask what it already knows. This is one of the clearest signals that the app is paying attention, and it is the difference between a companion and a form.

3.9 Onboarding

A first-run sequence that welcomes, orients, gates, and listens:

1. **The Initiation.** The welcome statement, personalized with the user’s name.
2. **Four reads.** The philosophy, the mission, the creator, the lineage. Available permanently afterward rather than only at first run.
3. **Disclaimer and Terms.** Both must be explicitly acknowledged. This gates all further progress.
4. **Discovery questions.** Thirty long-form questions on ego and Higher Self, individuality, purpose, conditioning, memory, love, prayer, and transcendence. Resumable by design, since the source preamble explicitly asks the user not to rush them.
5. **Begin the journey.** Hand-off into Module 01.

Discovery answers form the user’s baseline. Everything the app later says is more targeted because of them.

3.10 Module structure

A module is a place the user dwells, with a chat companion alongside the content. **Values is the worked example and the flagship**, and its shape is the pattern later modules follow. It arrives in release 2 rather than release 1 (§10):

- **Orientation.** Four lessons framing values as an inner compass rather than goals, through psychological, spiritual, and energetic lenses, closing on why unexamined values get inherited by default.
- **Discernment.** The user explores a 265-value library and color-codes what resonates: green (confirmed), yellow (wavering), red (let go of), purple (future self). Reflection sets then interrogate each color. The yellow and red sets ask where the value came from, whether it still serves, and what purple value might replace it. The purple set asks who the user is becoming.

- **Integration.** Narrowing to a top ten, then a top five. An alignment audit across relationships, self, career, home, family, habits, and choices. A closing reflection with a twelve-minute letting-go and gratitude meditation.
- **Baseline saved.** The values profile persists so future sessions can compare against it.

Any value in the library can be opened for a deep exploration covering etymology, psychological meaning, shadow expression, how trauma masquerades as it, how to tell an inherited value from a chosen one, and an alignment test. Sixteen of these are written.

3.11 Module visual surfaces

Chat is the connective tissue rather than the whole experience. **Each module offers something to look at and work with:** a surface where the thinking becomes visible and manipulable.

Some of this work is not conversational. Sorting through 265 values is an interface problem, not a dialogue: a field to range over, mark up, narrow, and stand back from. Being told your top five is different from seeing them arranged as something you made. The artifact is also what persists: the object a user returns to months later, the one they screenshot, the evidence that the work happened.

Values already implies its own set: the library laid out for browsing and four-state marking, the narrowing from ten to five, the alignment audit across the areas of a life, the finished values profile as something closer to a keepsake than a results screen, the meditation timer, and the reader for a deep exploration.

Others suggest theirs. Lotus of Life is inherently radial, with petals for the areas of a life, each showing how nourished it feels. Hawkins is a scale. Boundaries wants a map of where lines are needed and where they are missing. Shadow Work needs something that can hold what a person would rather not look at directly.

Most of them are not yet known, and this document does not pretend otherwise. Each module's artifact should be discovered as that module is built, with Values as the first proof, and early attempts should be expected to be wrong. What matters now is not specifying them but ensuring the architecture can accept them.

What that requires. A module cannot be a fixed template with content poured in. It has to **bring its own interactive surface** while sharing the plumbing every module needs: slot reading and writing, progression, persistence, and the chat companion. The contract between framework and module should therefore be narrow and stable. A surface renders, reads and writes slots, reports progress, and **tells the companion what the user is currently looking at.** That last part is what makes the conversation feel present rather than parallel: "you've marked forty values green; shall we look at which of those are actually yours?" is only possible if the agent can see the canvas.

Proposed: rather than sixteen bespoke surfaces, build toward a small vocabulary of reusable interaction patterns (select-and-mark, rank-and-narrow, radial or spatial map, scale, timeline, reader, timer) that modules compose. Every genuinely novel surface is also work repeated for iOS and Android later, which argues for a shared vocabulary over inventing freely each time.

Two constraints from the outset. Meaning cannot rest on color alone: the four value states are named as colors, and they need labels or shapes alongside them to be usable by anyone who cannot distinguish them. And these surfaces should feel contemplative rather than gamified. Her visual language is sacred geometry, fine line work, and muted ground; an artifact that looks like a progress dashboard would contradict §1.

3.12 The user profile

Everything the user does (discovery answers, module work, ad-hoc conversation) accumulates into a picture the app reasons from. Two properties matter:

- **It is a timeline, not a snapshot.** Answers are versioned and timestamped, so the app can say “last time you said security mattered more than wealth; is that still true?”
- **It is honest about how it knows.** Something the user stated plainly is held with high confidence. Something inferred from a tangent is held weakly until confirmed. When a later conversation contradicts an earlier answer, that contradiction is a door to open rather than an error to correct.

The user can see and edit most of what the app believes about them.

3.13 Life situations

A picture of who someone *is* differs from a picture of what they are **currently living through**: the strained relationship with a sibling, the decision that will not resolve, the job that is ending, the parent who is getting older, the move, the diagnosis in the family, the thing at work that has no clean answer.

None of this arrives through a form. Nobody opens an app and fills in their difficulties. These surface sideways, in conversation, across weeks: an aside, a detail repeated, a subject approached and left three times.

So the app listens for them and holds what it hears carefully: who is involved, the setting, the forces pulling in different directions, the tensions and paradoxes the person is caught between, what they feel, how they respond, what they have already tried, what they believe about it, and what is genuinely still ambiguous.

Then it shows them back. A situation becomes something the user can look at and work with: the people, the place, the forces, their own responses, laid out and interactive rather than described in a paragraph. It is offered as a **situational synopsis** for confirmation, on the same terms as a chat synopsis (§3.16). This is what I understand to be going on; have I got it right, what have I missed, what does not belong.

That echo is not administration. Much of this work is not solving a situation but **changing someone's relationship to it**, and seeing the forces named and arranged is frequently where that begins. People rarely see the shape of what they are inside.

Imagined futures belong here too: the conversation someone is dreading, the outcome they fear, the version of next year they keep rehearsing. These occupy as much room in a life as things that have happened, and they belong in the same structure, marked as anticipated rather than actual. Named, they usually loosen, though §12 sets out the line between naming a fear and rehearsing it.

How situations are used. They inform which modules would serve (§3.7), and they give conversation a backdrop, so the app is talking to someone in the middle of their life rather than to an abstraction.

Consent governs all of it. The app asks before opening a situation (“would you like to look at the thing with your sister today?”) and **no is a complete answer**, not a prompt to try a different angle. Situations can be corrected, muted, closed, or deleted outright. They are also allowed to end: this is a record of what someone is living through, not a permanent archive of everything that has ever hurt them.

3.14 Guiding and teaching registers

The app has two registers. **Guiding** is gentle, empathetic, holding space. **Teaching** is direct, challenging, and probing, pushing past the comfortable answer. The authored content already models the second: “Do these values restrict you?”, “Would I still choose this if nobody rewarded me for it?”

Getting the switch between them right is a design problem rather than a settings toggle. See §14.

3.15 Practices

Alongside reading, reflecting, and talking, the app offers **practices**: short, repeatable things a user actually does, in three families.

- **Physical.** Breath, somatic attention, nervous-system regulation, movement. The source material is already careful here: the Maté lens throughout the value explorations treats the body as a source of information, and the somatic inquiry section asks what happens in the body when a value is honored and when it is violated.
- **Mental.** Contemplative inquiry, attention training, sitting with a question rather than resolving it.
- **Emotional.** Gratitude, letting go, self-compassion, grief.

A practice has an intent, a duration, guidance the user is walked through, an optional timer, and an optional check-in afterward covering how it landed and what came up. Practices reach the user three ways: attached to a module at the moment they belong, offered by an agent when the conversation calls for one, or chosen freely from a library.

One practice is already authored: the twelve-minute letting-go and gratitude meditation that closes Values, with its fifteen breaths, its gaze point, and its somatic prompts. Discovery question 15 contains a second in embryo, three days off social media followed by noticing what the body does on returning. Everything else is unwritten, and a practice library is the clearest new authoring requirement this creates.

No streaks, no badges, no completion pressure. The content asks users not to rush, and gamifying a contemplative practice would undo the thing it is for. Practices may be gently tracked so the user can see their own pattern; they are never scored.

Proposed: practices need contraindication notes. Breathwork and somatic attention can destabilize someone with a trauma history, which the source material understands well, and the app should be able to say so and offer an alternative rather than pressing on.

3.16 The journey record

The journey is where the app’s account of the work and the user’s own reflections sit together, in time order. It replaces what earlier drafts called a journal: rather than a separate notebook, the record of what has been discussed *is* the journal, with the user’s own writing in the same stream.

A session is the unit. Each entry carries a date, a one-line summary of what that session was about, an account in her voice of what was actually said, what came out of it, and which modules were touched. The prototype adds a useful count to each: how many **actions**, **insights**, and **tensions** the session produced, so a user can see at a glance whether a sitting resolved something or opened something.

Three things fill it.

- **Session synopses**, written by the app after a conversation of substance.

- **Module reflections**, which land automatically. The yellow, red, and purple answers, the alignment audit, and the closing reflection are already journal entries in all but name.
- **The user's own entries**, written whenever they want, unprompted and unstructured.

A synopsis is a draft rather than a record. The user approves it, edits it, or asks for a different one (“that’s not what I meant”, “shorter”, “you missed the part about my father”), and only then is it kept. Nothing enters the record over the user’s head.

Proposed, and worth a decision: approving a synopsis is a strong signal and could be the moment the app hardens what it inferred during that conversation from weakly-held to confirmed. Correcting one would then be a correction to the profile as well as to the text. This would make the journey record the user’s main lever over what the app believes about them, which is a better mechanism than a settings screen full of fields.

Newest first, with what is waiting pinned at the end. The most recent session opens by default; one entry is expanded at a time. A signpost for what comes next sits at the bottom, because it is where the user is going rather than the latest thing they did, and it is not counted as a session.

The journey and the map answer different questions. The map (§3.5) is the sixteen modules and where the user stands among them, which is a question about place. The journey is a question about time: what has actually been discussed, what came of it, and what is still open. Coverage (§3.6) is read from the map; continuity is read from the journey.

The record is searchable, filterable by module or by kind of outcome, and exportable.

3.17 Inbox

Insight rarely waits for a session. Something surfaces on a walk, in an argument, at three in the morning, and by the time the user next opens the app it has gone.

The app therefore has an **inbox**. The user emails it or messages it on WhatsApp, and whatever they send (a thought, a situation, a photo of a page, a voice note, a one-line insight) is held for them. When they next sit down for a session, the inbox is waiting: “you sent three things this week; want to work through them?”

This inverts the usual relationship with an app. Instead of the app interrupting the user’s life for attention, the user drops things into it as life happens, and the work of making sense of them is done deliberately, in a session, when there is room for it.

Items can be worked through one at a time, written into the journey record, routed into the module they belong to, or acknowledged and cleared. What the user sends also feeds the profile, under the same visibility rules as anything else.

Two things this demands. An inbox is **not monitored in real time**, and the app must say so plainly at the point of sending, because someone in crisis must not believe a message has reached a person. Crisis language arriving through this channel needs an immediate automated response pointing to real help. And inbound email and WhatsApp are **unverified channels**: an address or number has to be bound to an account deliberately, or anyone who learns it can write into someone’s record.

3.18 Contact with Lelañea Fulton

The app is built from one person’s work, and users will want to reach her: sometimes with feedback or a passage that does not land, sometimes with a question the agent cannot answer, sometimes with something that moved them and wants to be told to a human being.

Share with Lelañea is available wherever the user is working. A module conversation, a journey entry, a reflection, a value that broke something open: any of it can be marked to share, with a note attached. It arrives in her review queue with the context she needs to understand it (§7.1).

Beyond sharing, there is a **messaging section**: an asynchronous conversation between the user and the real Lelañea, extending at its fullest to booking a 1:1 session with her, so the app becomes the front door to the practice it was built from rather than a replacement for it.

Subscriptions come later, so release 1 cannot gate this by tier. Sharing is the part that belongs in the early product: it costs her attention rather than her time, and it is where the most useful signal about the app will come from. Messaging and session booking wait for the commercial model, and the gating mechanism should be built as a capability check from the start, so introducing tiers later is configuration rather than surgery.

This needs care in three directions. It is **not a support channel and not crisis cover**, so response times must be stated honestly and set low, because she is one person. Sharing must be **explicit and revocable**: the user chooses exactly what goes, sees what she will see, and can withdraw it. And a direct exchange with a Professional Certified Coach is a **different relationship** from using an app. ICF ethical obligations and probably a separate agreement attach to it, which is a decision for her and a lawyer rather than for the build.

3.19 Data visibility and deletion

The app accumulates an unusually intimate record: someone's values, their conditioning, their relationships, their health, their fears, their childhood. That deserves a standard well above the ordinary.

Everything the app holds is visible, including the chat history behind the profile and the provenance of each conclusion: this is what you said, this is when, this is what I concluded and how confident I am.

Chat history can be archived, viewed, and deleted: a single exchange, a session, a module's worth, or all of it. Deletion includes what was derived from the deleted material, not only the transcript.

Export gives the user everything in a readable form, because a record of your own development should not be hostage to a subscription.

GDPR is a floor rather than a target. Users will be in Europe, so access, rectification, erasure, portability, and restriction are requirements. Much of what the app holds (health, sexuality, religious and spiritual belief) is **special category data** under Article 9 and needs an explicit lawful basis rather than an assumed one. Sending journey and conversation content to a third-party model provider is itself processing and needs the contractual and disclosure trail that implies. Retention periods, deletion propagation into derived data and embeddings, and where data physically sits are all decisions to make deliberately rather than to discover during a request.

3.20 Token budget and cost visibility

Every conversation costs something to run, and the app is straightforward about it rather than hiding it behind a subscription.

Ordinary use carries a running total in dollars. Chat, module work, and reflections are metered visibly and unobtrusively. An ordinary session costs a few cents, which is worth showing rather than hiding: the numbers are small, and seeing that is reassuring rather than alarming.

Expensive actions are quoted before they run. Convening the advisory council (\$5) or asking for a retrospective across a year of the journey record costs materially more than a reply. The app says so first, with a rough figure and a confirmation, so nothing significant is spent on the user's behalf without them choosing it.

In time the user sets their own budget for a period and spends it deliberately: this much this month, and I would rather spend it on depth here than breadth there. Deciding consciously where your resources go is the same discipline the modules teach, applied to the app itself.

In release 1 there is nothing to pay, so this is visibility plus a ceiling rather than a wallet. The framing still matters: the point is honesty about what is happening rather than restriction. When a ceiling is reached it should be met with an explanation and a choice, never a conversation cut off mid-sentence.

The unit is US dollars. Tokens are jargon and an invented unit would read as a game currency, so the app shows real money even while nothing is charged: what this month has cost to run, what is left against the budget, and what a longer piece of work is likely to cost before it runs. Honest, legible, and already the right unit when charging does begin.

4. Content assets

Six authored JSON files, delivered alongside this document in `content/`. This is real content, transcribed from Lelañea's own documents and corrected only for typography. It is not a draft for the build to improve on.

File	What it holds	State
<code>lelanea_foundational_documents.json</code>	Welcome, philosophy, mission, creator, lineage, disclaimer, terms. Each tagged with the app surface it belongs to.	Complete; legal placeholders open
<code>lelanea_module_structure.json</code>	The 17-module journey: onboarding plus 16 modules across 5 tiers	Module 01 fully built out; 02 to 16 are stubs, deliberately
<code>onboarding_discovery_questions.json</code>	30 discovery questions, two with yes/no branches	Complete
<code>values_module.json</code>	The Values module: 4 color states, 265-value library, 10 authored steps	Complete, and the flagship
<code>values_reference_framework.json</code>	The 22-point editorial brief behind the value explorations	Reference, not user-facing
<code>value_explorations.json</code>	Long-form deep-dives, one per value, on a common 25-section schema	16 of 265

Two content realities to design around. Fifteen of the sixteen numbered modules have a title and nothing else, so the skeleton must be visible and navigable without pretending to be finished. And 249 of 265 values have no exploration written, so the app needs a defined, graceful fallback when a user opens one.

The authoring pipeline is the rate limiter on everything after release 1. Since release 1 carries no module content at all, everything a user eventually works through depends on it. Once Values proves the pattern, what stands between the app and sixteen working modules is writing, not engineering. That deserves treating as a real workstream rather than an assumption:

- **A defined shape for a module's content.** Values is the template: lessons, an exercise with a library, reflection sets, a distillation, a saved artifact. Whether every module fits that shape or each needs its own is unknown until the second one is attempted.
- **A drafting workflow with her,** since the content has to be hers. Ghost-writing to a template would produce material the voice fingerprint then has to paper over, which is the wrong way round.
- **Review, sign-off, and versioning,** with content shipping without a redeploy (\$9, Operations).
- **A measured cycle time for one module,** taken early, because it sets the roadmap for everything after release 1 more than any engineering estimate will.

5. Architecture

Lelañea is a **leaf fork of the Daybreak framework**, which was designed with transcendental coaching as its named reference case. The technical grounding for this section is [reference/lelanea-app-spec.md](#), with the non-technical account of the five concepts in [reference/Lelanea_Framework_Overview.pdf](#). The five concepts in the framework overview map onto Daybreak primitives that already exist, so this is assembly rather than invention.

Concept	What it is for Lelañea	Daybreak primitive
Modules	The places the user dwells: Values, Boundaries, Standards, and the rest	<code>ModuleDefinition</code> , registered via <code>registerModule()</code>
Facilitation	The map and the rules: a recommended spine with open navigation, always-open tangents, and recency-weighted recommendation	<code>FacilitationGraph</code> plus engine and guidance layer, bounded by <code>FacilitationPolicy</code>
Data-slots	Everything learned about the user, versioned, with confidence and provenance	<code>SlotDefinition</code> and <code>SlotValue</code> , scoped <code>global / module:<slug> / facilitation; mode: open</code> for qualitative material
Knowledge base	Her workbook, podcast, articles, and the value library: what the agent speaks <i>from</i> , paired with the voice fingerprint below, which governs how	Sunrise Knowledge Base plus agent knowledge grants, scoped per module and tagged per value
Agents	The voices the user meets, the helpers behind them, and the advisory council	Facilitation agent roles (<code>onboarding</code> , <code>synopsis</code> , <code>orientation</code> , <code>state</code> , <code>path/progress</code> , <code>facilitator</code>) plus module-scoped agents

The slot taxonomy the framework overview sets out is carried forward as-is: *life areas* (physical health, emotional health, wealth, relationships, family, work, spiritual health, including where things feel dysfunctional and where they are working); *the person* (personality, life patterns,

psyche, aspirations, blind spots); *external conditions* (their situation and its constraints); *preferences* (how they respond to certain kinds of question). Module output such as values, standards, and boundaries is held as slots too, so work done inside a module travels with the user rather than being stranded there.

The taxonomy has to be large, and it has to be data. What genuinely helps here is broad: personality and disposition, health of every kind, the stories someone tells about themselves, relationships and family systems, profession and working life, formative experience, current constraints, and where someone sits in their own development. That last one matters and is the most delicate. The app benefits from understanding whether someone is early in examining their conditioning or has been at it for decades, because it changes the pace, the register, and what will land. It is a tuning signal, never a grade (§12). A taxonomy this size cannot live in code: slot definitions need to be authored, uploaded, versioned, and revised by admins as understanding improves, with the app reading them as content in the way it reads Lelañea's writing.

The agent family, likewise, follows the framework overview: a Consultation and Discovery agent that builds the first picture and mirrors the person's own words back; a Life Situation agent with sub-agents for specific circumstances; specialist agents per module; a Curator that surfaces the right piece of her material; a Supervisor that watches how the others treat the user.

The boundary. Work goes in `lib/app/*` and `modules/<slug>/`. Nothing touches `lib/framework/`. Where a genuine seam is missing, it gets flagged upstream rather than patched from the leaf.

Voice fingerprint

The framework overview names five concepts. This app needs a sixth, and it may be the most valuable thing built here.

The knowledge base is what she knows; the fingerprint is how she says it. Two different artifacts, both required. Without the knowledge base the app invents content. Without the fingerprint it delivers her content in the voice of a generic assistant, which is the worse failure, because content is the part that is easy to source and voice is the part that makes it hers.

Built from her corpus: podcast transcripts, articles, Substack pieces, the workbook, talks, and, with the care set out below, 1:1 client transcripts.

What is extracted is manner rather than matter:

- Recurring phrases, constructions, and the shapes her sentences take, including the single-line cadence that runs through her written work.
- Vocabulary: the words she reaches for, and just as tellingly the ones she avoids.
- The metaphors, analogies, and images she returns to.
- How she opens, how she closes, how she leaves silence.
- Where she hedges and where she is absolute, and how she couches something difficult before saying it.
- How she grounds a claim: citing lineage, telling a story against herself, or turning it back as a question.
- Her ethical lines: what she declines, and how she declines it.
- Her humor, and where it is and is not allowed.

It is a repertoire rather than a single voice. She is different coaching and teaching, different when someone is grieving and when someone is intellectualizing a feeling to avoid it, different when direct and when deliberately indirect. The fingerprint is therefore a set of **categorized voice**

profiles plus rules for which applies when, and contextual selection is a first-class feature rather than a refinement.

Proposed representation, since it has to survive contact with a prompt: a layered artifact rather than one blob. A small **always-on core** (identity, cadence, the hard nos) that costs little and is never omitted. **Context-selected overlays** for register and situation, chosen by the same guidance machinery that decides everything else. And **retrieved exemplars**: actual passages of hers, pulled at prompt time from the same vector store as the knowledge base. Showing a model how she says something beats describing it, and exemplars keep the fingerprint tied to real sentences rather than to somebody's characterization of her.

User preference is a filter rather than a fourth layer of authorship. The leanings of §3.4 modulate selection and weighting within the fingerprint (which overlays apply, which exemplars are retrieved, how much elaboration), bounded so that no setting can reach the invariant core. Some dimensions are user-adjustable and some are hers alone, which is an authoring decision made in the workshop rather than a UI decision made later.

This is ongoing rather than a setup task. Every episode she records and every piece she writes can extend it. It is also the asset the framework carries forward, because the next expert-led app needs exactly this, built the same way.

Recommendation mechanism

The question in §3.7, what would genuinely serve this person now, is answered by two mechanisms doing different jobs.

Deterministic rules bound the space: what has been acknowledged, what has been done, what must not be offered yet, what is unsafe to suggest, and how recently something was declined. These are cheap, fast, testable, and they always run. Shadow Work is not proposed to someone on their second day, nothing is recommended before the terms are accepted, and a module declined last week is not pushed again this week.

Semantic evaluation ranks and explains within that space. Agents read the fuller picture (the profile, the last few sessions, recent journey entries and inbox items) and judge what would serve, weighted toward the recent. This is where a decision someone has circled three times gets noticed, which no rule would catch.

Proposed: every recommendation carries its reason as data rather than as prose generated after the fact. That is what makes the promise in §3.7, ask why and get an honest answer, real rather than a plausible-sounding explanation invented on request. It is also what makes recommendation quality auditable (§7.2).

Advisory council

Some situations deserve more than one lens. A **team of perspective agents** can be convened on a user's situation. Each is given a synopsis and the relevant, consented parts of the profile, and each evaluates from its own tradition: Jungian shadow and individuation, Stoic control and acceptance, nondual inquiry, somatic and nervous-system, parts work, values and standards. Their readings are then blended, with the perspectives kept **distinguishable rather than averaged together**, because the disagreements between them are frequently the most useful part.

The council is invisible as machinery and present as conversation. The user does not meet six agents or read six reports. Lelañea introduces the perspectives herself, in her own voice, as a coach with a wide reading list would: "Carl Jung might read the figure in that dream as something you have disowned rather than something outside you." "A Stoic would ask which part of this is

actually yours to decide.” She can hold two of them against each other and say which she finds more useful here, or leave the tension open. The council is a way of widening what she can bring to a situation, not a panel the user is handed.

Three things this needs to get right. It is **expensive**, so it is quoted before it runs (§3.20). It is **fallible in a specific way**: a Jungian lens can drift into caricature, so each must be grounded in real material and signed off by Lelañea rather than improvised from a model’s general impression of Jung. Attribution also has to stay honest, since “Jung might read this as” is a claim about Jung. And it is **offered as perspective rather than verdict**: several ways of seeing a situation, handed to the user, who decides. That is the principle of §3.7, applied where it is hardest to hold.

Data storage

This is a genuine architectural decision, because the two obvious answers are each half right.

Structured storage, relational, is what a slot needs to be: a value, a version, a timestamp, a confidence, a provenance, a scope. It supports “show me everything you hold, let me correct this one, delete that one”, which §3.19 makes non-negotiable. It is precise, auditable, and deletable.

Vector storage is what makes the app feel like it remembers. If everything a user has said is embedded, a conversation happening now can surface what they said eighteen months ago about their father, or the reflection where they first admitted a value was not theirs. That is the difference between an app that has a profile of someone and an app that knows them.

Proposed: both, with the relational store authoritative. Slots and their history live relationally and are the source of truth for what the app believes, shows, and deletes. Utterances, reflections, journey entries, and inbox items are additionally embedded for associative retrieval, with every embedding carrying the id of the record it came from, so a deletion request removes the vector alongside the row and the two cannot drift. Retrieval at conversation time draws on both: the structured picture for what is true about this person, the vector store for what they have said that resonates with this moment.

Situation ontology

§3.13 needs more structure than a slot. A situation is a small graph, and it needs a defined shape so it can be reasoned over, drawn, and corrected:

- **The situation itself**: a short title in the user’s own framing, a summary, a temporality (past, current, anticipated), and a lifecycle state (emerging, live, receding, closed).
- **Actors**: the people involved, held **by role rather than identity** wherever possible, such as *my sister* or *my manager*. The app is not building profiles of people who never consented to being in it.
- **Setting**: where and in what context this plays out.
- **Forces**: what is pulling in each direction, which is usually the part a person cannot see from inside.
- **Tensions and paradoxes**: the binds that have no clean resolution and should not be handed a tidy one.
- **Emotions and responses**: what it produces in them, and what they do about it.
- **Attempts, beliefs, and open questions**: what has been tried, what they take the situation to mean, and what is honestly unresolved.
- **Relations**: to values (“this collides with a value you marked green”), to modules, and to other situations, since the same pattern often recurs across a life in different forms.

Every element carries provenance, confidence, and a timestamp, on the same terms as any slot, so nothing about someone's life is asserted without a record of where it came from and how sure the app is. Structurally this is a layer over the storage above: the graph relational and authoritative, the underlying utterances embedded so a situation can recall what the user actually said about it eighteen months ago.

The ontology must be extensible as data, like slot definitions. Nobody will get this vocabulary right at the first attempt, and the useful version will be discovered by watching real conversations rather than designed in advance.

Localization

Users will be anywhere. The app should ask where someone is and, in time, what language they prefer, and be built so that adding a language is a content exercise rather than a rebuild.

In practice: no user-facing string hard-coded in a component; all authored content, including the six JSON files and everything that follows, carrying a locale and structured so a translation is a sibling rather than a fork; dates, times, and the meditation timer locale-aware; crisis resources and legal text regionalized, since those genuinely differ by country and are where getting it wrong matters most.

Worth naming early: translating Lelañea is not a normal localization job. Her cadence, her deliberate line breaks, and her particular use of Self and self would all be flattened by machine translation. Whether translation means human translators working with her, or an explicitly reduced experience in other languages, is a decision to make before it is urgent.

Platform strategy: desktop first, API-first

V1 ships as a desktop web experience, with native iOS and Android apps to follow.

Nothing built in v1 should assume the browser is the only client, because the cost of retrofitting that assumption is the entire experience layer.

The discipline this implies:

- **Every capability the web client uses is reached through a versioned API.** No behavior exists only inside a page handler or a server-rendered route. If the web app can do it, a native client can do it with the same call.
- **The web client is the first consumer of that API rather than a privileged one.** Treating it as one client among several from day one is what makes the second and third cheap.
- **Auth is token-based and client-agnostic**, because native apps cannot ride on browser session cookies.
- **Authored content is served through the API rather than compiled into the web build.** The six JSON files are the source; a mobile client must render the same copy without a parallel content pipeline, and a content correction must reach every client the same way.
- **State is server-side and authoritative.** Someone who starts Values on a laptop and continues on a phone must not lose a color-code, a reflection answer, or their place in the thirty questions. Versioned, timestamped slots already give this; the requirement is that no client ever holds state the server does not.
- **Chat streams over a transport both can use**, SSE or websockets, rather than anything browser-specific.
- **Responsive web reduces later rework**, but a phone browser is not the mobile deliverable. Native is.

- **Module surfaces render from API state** and hold no authoritative state of their own. A value marked green on a laptop is marked green because the server says so, not because a component remembers. Every distinct surface is also a second implementation on native later, which is the practical argument for the shared pattern vocabulary in §3.11.

The framework overview already anticipates meeting the user across desktop, iOS, Android, WhatsApp, SMS, and email. Those messaging channels are well beyond release 1, but they are the same argument in a different shape: each is another API consumer, and API-first is what keeps them from each becoming a rebuild.

Proposed, and worth checking early: Daybreak’s own client/server split needs testing against this before Values is wired. If its module and facilitation surfaces are server-rendered-first, the API boundary for a leaf app needs defining deliberately, ideally in phase 0 and certainly before phase 5, rather than discovered when the iOS work starts.

6. Design system

A starter design system has been supplied (`design/Lelanea_Design_System.zip`) and the build begins from it. It is explicitly a starting point rather than a finished specification, and it will change as real screens get made. The zip remains the source of truth for code; what follows is the part a builder needs to hold in mind.

Contents: `colors_and_type.css` (tokens), `styles.css` (global entry), seven React components under `components/`, a mobile UI kit under `ui_kits/lelanea_app/`, brand assets, preview cards, and a `SKILL.md` that is directly usable as a Claude Code skill. Components are exported on `window.LelaneaDesignSystem_a15360`.

6.1 The approved prototype

`design/lelanea.html` is the approved design prototype and the reference for how the product looks and behaves. It is accompanied by `design/lelanea-design-prompt.md`, the note that sets out how it should be read. It is a single self-contained HTML, CSS, and JavaScript file covering two surfaces:

- **The public site:** nav, hero with the lotus mark, waitlist form, “what this is and what this is not”, tiers, quote bands, footer.
- **The signed-in app:** a four-column shell, plus the journey view, module views, situations, resources, usage and billing, settings, and account.

Precedence. The prototype governs how the product looks and behaves: layout, information architecture, theming, type, motion, and interaction. This document governs what the product is and what it says. Where the two disagree on concepts, vocabulary, or content, this document wins, because the prototype is a sketch of the possible rather than a specification of the substance.

It is a look-and-behave specification rather than an implementation. The build is on a different stack, so what carries across is the layout and information architecture, the palette in both modes, the type, the motion, and the interaction behavior: how panes resize and collapse, how drawers open over content, and how the navigator shows what is done and what is current.

It is also the source of the initial content. The public-site copy is real and considered, and should be reused as written. Inside the app the copy is a deliberate skeleton: the structure and the module names are real, the module bodies are placeholders, and they should not be mistaken for finished content.

Two views in it are worth naming because they settle questions this document had left open. **Settings** is titled “how she speaks to you” and renders the leanings as eleven sliders with the framing “filters over her voice, not replacements for it”, which matches §3.4 exactly. And **the journey view** is the shape §3.16 now describes: sessions newest first, one open at a time, each with what was said, what came of it, the modules touched, and counts of actions, insights, and tensions.

The prototype also introduces one surface this document did not have: a **Resources** drawer of films and reading. That is the Curator agent of §5 given a home, and it should be treated as a real part of the product rather than decoration.

6.2 Color

A warm, earthy palette tempered by meditative blues. Burnt orange is the ceremonial accent, used for the lotus center and the primary action and very little else. Deep teal and light aqua are the lotus petals and carry navigation, active states, and iconography. Backgrounds are oyster white in light and near-black charcoal in dark, never pure white or pure black.

Role	Light	Dark
Background	Oyster white #F3F0EC	Near-black charcoal #282C2E
Surface	Dusted stone #EBE6DF	Mid dark steel #3A3F42
Surface, sunk	#EEE9E2	#323638
Heading text	Near-black teal #11181A	Oyster white #F3F0EC
Body text	Near-black charcoal #282C2E	Warm soft stone #E3DAD1
Secondary text	Slate ash #5A5F62 (see note)	Mist ash #A2A7AA (see note)
Border	rgba(111,115,118,0.24)	rgba(227,218,209,0.12)
Divider	rgba(111,115,118,0.16)	rgba(227,218,209,0.08)

Brand and functional colors hold across both modes: accent burnt orange #C96F43 (pressed #B5633B), brand teal #17718A, brand aqua #7CC0D6, success sage forest #457B6A, danger terracotta red #B75D52, warning dusted ochre #C9A65D, info steel slate blue #497AA8, selected heather amethyst #806C7B. Three further accents sit in the palette for wider use: dusted coral #BC8A76, muted lavender grey #C4B3BE, and mid teal #3E96AE.

One proposed change to the supplied tokens. The kit uses neutral silver ash #6F7376 for secondary text in both modes. Measured, it reaches 4.21:1 on the oyster background and 3.85:1 on the stone surface in light mode, and 2.95:1 and 2.23:1 in dark. All four fall short of 4.5:1, and secondary text is where meta lines, timestamps, hints, and helper copy live, so it is used constantly. Two replacements hold the same warm neutral character while clearing the threshold on both the background and the surface: **#5A5F62 in light** (5.69:1 and 5.21:1) and **#A2A7AA in dark** (5.80:1 and 4.39:1, the latter close enough to raise to #A8AEB1 at 4.75:1 if secondary text is used on dark surfaces often). Keep #6F7376 in the palette for borders, dividers, and disabled states, where contrast is not carrying meaning.

For reference, the rest of the palette measures well: body text is 12.41:1 in light and 10.21:1 in dark, and brand teal reaches 4.91:1 on oyster. The one to watch is the burnt orange accent at 3.17:1, which is fine for a button fill or a mark and should not be used for small text on the background.

Light and dark are equal citizens. Tokens switch on `prefers-color-scheme` and on an explicit `[data-theme]` attribute, so a user preference can override the system setting. Neither mode is a tint of the other, and every surface should be checked in both rather than designed in one and converted.

6.3 Typography

- **Display: Instrument Serif.** Hero moments, section headers, single-sentence prompts. Set tight, `line-height: 1.05`, `letter-spacing: -0.02em`, and given room to be large: 40 to 72px.
- **Body and UI: Hanken Grotesk.** 16px base, `line-height: 1.6`, weights 400, 500, 600, never bolder than 600.
- **Quote italic: Cormorant Garamond.** Pull quotes, session prompts, invocations. This is the register that reads as her own voice on the page.
- **Numerics** tabular wherever alignment matters.

Scale: display 72 / 56 / 40, then h1 32, h2 26, h3 20, body large 18, body 16, body small 14, meta 12. Line heights 1.05 display, 1.2 heading, 1.6 body, 1.4 meta. Tracking -0.02em display, -0.01em heading, and 0.14em on the lowercase tracked-out eyebrow labels.

6.4 Spacing, radii, elevation

Spacing runs on an 8-point scale: 4, 8, 12, 16, 24, 32, 48, 64, 96, 128. The system prefers the larger end. Minimum 24px gutters on phone, with 16px reserved for tight inline groupings.

Radii: 12px default, 20px cards, 28px large, 999px pills and the lotus action. No hard corners anywhere except full-bleed sections, which have none.

Borders are hairline 1px at 24% alpha in light and 12% in dark.

Elevation has two layers only. `--shadow-rest` for resting cards, `--shadow-lift` for modals and the active lotus halo. No inner shadows. No colored shadows except the lotus center, which casts a soft orange bloom (`--shadow-bloom`).

Cards: surface color, 20px radius, resting shadow, 20 to 24px internal padding, and no border by default. A border appears only in dark mode, at 8% alpha, to separate the card from the near-black background.

6.5 Motion

- **Easing** is `--ease-breath`, `cubic-bezier(0.22, 0.61, 0.36, 1)`, a gentle breath-like ease-out. `--ease-quick` exists for utility transitions.
- **Durations:** 200ms for micro-interactions, 420ms for transitions, 1200 to 2400ms for ceremonial moments.
- **Fades over slides.** Opacity is preferred to translation. Where translation is used, 4 to 8px is the limit.
- **No bounces, springs, or overshoot.** The product breathes; it does not perform.
- **Hover** deepens a background by about 6% in light mode or lightens it by about 6% in dark, never shifting hue. Text links gain a 1px underline at 3px offset with no color change. The primary orange button dims to `#B5633B` rather than brightening.
- **Press** is `scale(0.98)` over 120ms on the breath easing, with the background flattening one step and no color shift.

6.6 Components

Seven components ship in the kit, each with a `.d.ts` and a preview card.

Component	What it is
Lotus	The signature bloom. Three tiers of petals that fan open over 2.2s, then breathe. Props: <code>size</code> , <code>open</code> , <code>autoOpen</code> , <code>idle</code> , <code>water</code> , <code>delay</code> , <code>onOpened</code> .
LotusMark	Static lotus glyph for avatars, favicons, and inline marks. Pass <code>water={false}</code> under about 40px for a tight crop.
Button	Pill button in <code>primary</code> (burnt orange), <code>secondary</code> , <code>ghost</code> , <code>destructive</code> .
Card	Elevated stone surface with optional lowercase eyebrow, serif title, body, and meta line. Supports dark.
ChatBubble	One turn in a sit. <code>from="ai"</code> renders stone with the lotus avatar; <code>from="user"</code> renders deep teal, right aligned.
Chip	Theme or filter tag. <code>selected</code> renders heather amethyst; <code>tone="teal"</code> marks an active teaching.
Banner	Quiet system-state banner in success, error, warning, or info.

On both lotus components, `size` is the rendered width of the bloom rather than of the SVG frame, so the two are interchangeable at the same size.

6.7 Iconography

Lucide is the baseline family, chosen for its 1.5px stroke, rounded caps, and open geometry. Icons should read as small drawings rather than wayfinding glyphs: 1.5px stroke, never filled, never bicolor, sizes 16, 20, and 24 with 24 as default, colored from secondary text at rest and deep teal when active.

Custom marks are the lotus glyph (favicon, app icon center, a 16px mark above pull quotes, the idle state of the home animation) and the logo lockup.

No emoji, ever. The thin bullet `•` is allowed in meta text such as `4 min • yesterday`. A single lotus glyph may act as a poetic divider where the SVG cannot render.

6.8 Layout and surfaces

- **Fixed elements:** top status bar, top nav, the chat input, and the tab bar. Everything else scrolls.
- **Single column on mobile**, with content capped at 560px on larger surfaces. Safe area insets respected.
- **Backgrounds are flat color.** Never full-bleed photography, no repeating patterns, no grain overlay, and no hard gradients on UI elements. Gradients belong to the lotus and to a soft radial halation behind it, roughly 600px across at 8% opacity maximum.
- **Capsules over scrims.** Floating controls sit in solid capsules with the resting shadow. The single exception is the chat input, which uses a 64px protection gradient above it so scrolling text fades out.

- **Backdrop blur on two surfaces only:** the top status bar over scrolling content, and the chat input capsule, at `blur(20px) saturate(140%)`. No frosted cards and no translucent modal backgrounds; use solid surfaces with a dim scrim.
- **Imagery**, if used at all, is warm, naturally lit, and earthy: linen, clay, dried flowers, morning light, hands, water. Desaturated 10 to 15%, never hyper-saturated, no stock-business imagery. Illustration is preferred to photography in-product.

6.9 The lotus

The signature element. A bloom of three petal tiers over sage ripples, with a burnt orange core.

- **Outer tier:** 6 petals in deep teal `#17718A`, 60 long by 48 wide, at $\pm 66^\circ$, $\pm 46^\circ$, $\pm 25^\circ$.
- **Mid tier:** 6 petals in mid teal `#3E96AE`, 78 by 50, at $\pm 42^\circ$, $\pm 26^\circ$, $\pm 11^\circ$.
- **Inner tier:** 5 petals in light aqua `#7CC0D6`, 98 by 52, at $\pm 30^\circ$, $\pm 15^\circ$, 0° , with pale center veins.
- **Core:** burnt orange radial gradient with a soft bloom shadow.
- **Ripples:** three concentric sage rings reading as stillness on water rather than foliage.
- **Opening:** 2200ms, staggered inner to outer, from a near-upright cluster out to rest.
- **Idle:** a 4s breath, scaling between 1.0 and 1.015.

It opens **once per session** and is ambient thereafter. It is the first thing a new user sees, and the animation is the app's opening gesture rather than a loading state.

6.10 Copy and casing

The design guide sets rules for product copy, and they hold except where the voice fingerprint (§5) says otherwise. The fingerprint governs what the app says because it is derived from her actual corpus; the design system governs how it looks. Where a rule here is about form rather than voice, it applies.

- **Sentence case** for labels, buttons, nav, and body. Never all-caps, never Title Case on buttons. Eyebrow labels may be lowercase and tracked out.
- **No exclamation points.** Even "Welcome!" becomes "Welcome."
- Ellipses sparingly, only where a pause is genuinely intended. Periods optional on single-line labels.
- **Second person.** The app says "you", never "the user". Her teachings are referenced in the third person: "Lelañea's practice of return".
- **Questions over answers**, which matches the authored content, where most of the work is carried by questions.
- **Grounded rather than mystical-performative.** Words like presence, witness, arrive, and return belong. Stacked mystical vocabulary does not.
- Button labels follow the same discipline: `Begin` rather than "Start Now", `Return` rather than "Back", `Close` rather than "Dismiss".

Sample states from the guide, which are a useful calibration for anyone writing UI copy: *"What would you like to bring in today?"* as an empty chat state, *"Something didn't land. Try that once more."* as an error, *"Rest here for a moment before you go."* at the end of a session.

6.11 Gaps and substitutions

The kit is mobile and iOS-flavored; v1 is desktop web. The patterns this product needs and the kit does not yet contain are the two-panel layout of §3.3, the working surface on the right, the

per-turn disclosure drawer, the coverage panel of §3.6, the budget display, and the module visual surfaces of §3.11. These should be built as extensions of these tokens rather than as a separate desktop language.

Three substitutions are flagged by the kit itself. The fonts are Google Fonts chosen to carry the intended mood rather than a supplied specification. Lucide is a stand-in icon family. And the supplied wordmark reads “Lelanea” without the tilde, which does not match the agreed app name. All three need confirming or replacing (§14).

Two constraints here reinforce arguments made elsewhere in this document from a different direction: no gamification, and no emoji. The source worksheets use emoji in their lesson headings, and the authored JSON already carries icon names instead, which turns out to be the right form for this system.

6.12 Where the prototype and this document disagree

Flagged rather than silently resolved.

Billing appears in the prototype and is deferred here. Its “usage and billing” view shows a monthly budget in dollars, a running total against it, spend by day and by week, and a saved payment card. **The dollar unit is adopted** and is now the answer in §3.20. What does not ship in release 1 is the saved card and the user-set monthly budget, both of which belong with the later commercial phase. The spend views themselves are release 1 material, because the metering exists from the first agent call.

The public site shows tiers. Same tension. The copy is real and useful, but publishing tiers before there is anything to buy sets an expectation the product cannot meet in v1.

“Module closed” is wrong and the document governs. The prototype’s journey view counts modules closed and its map marks modules done. A module is never complete (§3.6), so that vocabulary does not carry over. **Sessions close; modules do not.** A session can be closed, a sitting can be finished, and a value can be decided, but a module is only ever more or less covered, and always open to being returned to. The prototype’s layout for these views stands; its wording is superseded.

Three features have no design yet: the inbox (§3.17), practices (§3.15), and the advisory council (§5). The council in particular needs care, because §5 says it should be delivered conversationally in her voice rather than as a visible panel, so its absence from the shell may be correct.

One content nuance to keep. In the prototype the user objects to “let go of” as a name for the red mark, on the grounds that it sounds like a verdict. That is a note about the authored content in `values_module.json`, where the four states are named from the source worksheets, and it is worth putting to Lelañea.

7. Operator surfaces

Two audiences other than the user: **Lelañea herself**, and whoever runs the platform. They need different things and should not share one screen.

7.1 Lelañea’s console

Her side of the contact described in §3.18, plus the means to speak to everyone at once.

The shared queue. Everything users have chosen to send her, in one place: a module conversation, a set of values answers, a journey entry, a chat excerpt, a piece of feedback, a suggestion. Each arrives with enough context to make sense of it and a clear statement of what the user consented to share. She can respond or simply acknowledge, and items can be marked read, kept, or cleared.

Direct messages. Threaded, asynchronous conversations with users: with everyone in the early product, since there are no tiers yet, and later with those whose tier includes it. A context panel sits beside each thread showing where they are in the journey, what they have shared, and what they are working through.

The consequential decision here is what that panel contains. Lelañea seeing everything about every user by default would be wrong and would undercut §3.19 entirely. The defensible position is that her view is **consent-scoped**: she sees what a user has explicitly shared plus what they agreed she could see on entering a messaging tier, and nothing else. Anything beyond that should require the user's active choice, in the moment, on the thing itself.

Announcements. She hosts a free monthly workshop and runs two podcasts, and users will want to know. A place to post meetups, workshops, episodes, new modules, and seasonal offerings, targeted where it makes sense by tier, locale, or position in the journey, and scheduled rather than sent by hand.

Delivered in-app, or by email or WhatsApp where the user has opted in. *This is the feature most able to damage the app's character.* An app that asks users not to rush cannot then push at them weekly. Announcements need frequency caps, per-channel opt-outs honored without argument, and a bias toward the user finding them on return rather than being chased.

Feedback. Suggestions and confusions gathered and themed, so that a passage users repeatedly misread becomes a content fix rather than thirty separate replies.

7.2 Admin dashboard

Growth: signups, waitlist conversion, activation, how many return and how often.

Journeys: where users are across the map, module starts and completions, and drop-off at phase granularity. That last one is the valuable part. Knowing whether people abandon Values at the 265-item inventory, at the third reflection set, or at the eighteen-question alignment audit tells you what to fix, in a way a completion percentage never does. The same applies to onboarding: which of the thirty discovery questions is where people stop.

Content: which values are chosen most, and in which color; which explorations are opened; which practices are used; which passages precede an exit. This is genuinely interesting to Lelañea as an author, not only to an operator. *Aggregate only, with a minimum cohort size before any figure is shown*, so that no view of what people value can be narrowed back to a person.

Cost: token consumption per user, session, module, agent, and model, with spend derived from it. **This is a release 1 requirement, and it matters more without revenue than it would with it.** There is nothing on the other side of the ledger yet, every conversation is pure cost, and the product's whole texture is open-ended conversation. Alerting on runaway sessions, and per-user ceilings that degrade gracefully rather than cutting someone off mid-thought.

The second purpose is evidence. When subscriptions arrive, the price and the shape of the tiers should be set from what real users consumed: how much a heavy month costs, how far the distribution spreads, what a completed Values module costs end to end. That means release 1

metering needs to be granular enough to answer pricing questions later, not only to raise an alarm now.

Safety and quality: supervisor-agent flags, crisis-detection events, refusals, complaints, and sampled review of conversations. Transcript access is the sharpest edge in the whole system. It should be exceptional, justified, logged, and visible to the user in their own audit trail, never a browsing convenience for staff.

Operations: deployed content versions, slot definitions, agent configurations, knowledge base ingestion status, error rates.

Roles and audit. Platform admin, Lelañea, and support are different roles with different reach, and every access to personal data is logged. The people running the app should be as accountable for what they can see as the app is to the user in §3.19.

7.3 Voice workshop

Where the fingerprint described in §5 is made and maintained.

Upload and designate. Every document arrives with a purpose set by an admin: knowledge only, voice only, or both. A marketing page may be neither. A client transcript may be voice-only and never quotable. Sensitivity and licensing are recorded at the same moment rather than reconstructed later.

Extraction with evidence. A pass over new material proposes candidate patterns, and **every candidate cites the passages it came from**. Nothing enters the fingerprint as an unsupported assertion about how she speaks. If a pattern cannot show its evidence, it does not go in.

Review and shape. Patterns are accepted, rejected, edited, or merged. Each carries a **weight**, how strongly it should press on generation, and a **scope**, which contexts it belongs to. This is where a pattern that is true of her podcast but wrong for a grieving user gets confined to where it belongs.

Contextual rules. Authored, readable, testable statements of which profile applies when: in grief, this; when someone is intellectualizing, that; in the first session, warmer and slower. These sit alongside the guidance rules in §5 and are edited the same way.

Adjustability bounds. Which preference dimensions users may move, how far, and what is locked. This is where Lelañea decides that someone may ask her to be plainer but not to stop asking questions, and that nothing may switch off how she handles distress.

Versioning and regression. A fingerprint version is a release. A standing set of prompts runs against the new version and the old, outputs side by side, and **Lelañea judges**, because it is her voice and no metric substitutes for her ear. Nothing ships unheard.

Drift watch. Sampled live outputs scored against the current fingerprint, flagging where the model has slid back toward generic assistant. Voice decays quietly and needs monitoring like any other quality.

8. Reliability, safety, and operations

Defaults rather than final answers, but defaults the build should start from rather than discover.

8.1 Failure and degraded mode

Most products can answer a failed request with an error toast. This one cannot: a user may be mid-sentence about their marriage when the model provider rate-limits, and a red banner is a rupture rather than an inconvenience.

The defaults:

- **Nothing the user typed is ever lost.** A message that fails to send stays in the box, retryable, with the conversation intact around it.
- **The app says what happened in plain language** and offers a choice. It does not spin, and it does not surface provider error text.
- **Reading survives generation.** Authored content, the values library, explorations, past sessions, the journey record, and the profile are served from storage and stay available when the model layer is down. A user can still read, reflect, write, and color-code values. Only generation stops.
- **Writes made during an outage persist and sync.** Journey entries, reflections, and slot edits are not held hostage to an agent being reachable.
- **Turns are idempotent.** A retried turn carries the same turn id, so a repeat cannot double-write the profile or bill twice.
- **Deadlines are explicit:** a first-token deadline after which the app speaks rather than waits, and a total-turn deadline. Both are configured rather than implicit in a library default.
- **Fallback models are allowed and always disclosed.** If the assigned model is unreachable and a fallback answers, the turn's disclosure drawer (§3.3) says so. The voice is never silently swapped.
- **The crisis path must not depend on the model.** Detection and response need a deterministic route, so an outage cannot remove the safety net at the moment it matters most.
- **Incidents are disclosed rather than hidden,** with a status surface and a banner where it affects use.
- **Data loss is the unacceptable failure.** Backups with a stated recovery point objective, restores actually tested. Losing someone's journey record is not comparable to losing a shopping cart.

8.2 Model strategy and evaluation

The provider plumbing comes from Daybreak. What models are used, by which agents, and when they change are Lelañea's decisions, and they need somewhere to live in the admin area (§7.2).

A model registry, editable by an admin rather than in code, holding for each agent: the model and its pinned version, a fallback, generation parameters, and the cost per turn it produces in practice.

Different agents deserve different models. Slot extraction, classification, routing, and summarizing do not need the strongest model. The coaching voice and the advisory council do. Getting this allocation right is one of the larger levers on cost, and it should be tunable without a deploy.

Version pinning, never "latest". A model upgrade is a change to the product's voice and judgment, and it should be a decision rather than an event that happens overnight.

Evaluation before promotion. A standing suite of golden conversations run against each candidate model and prompt combination, scored for voice fidelity (§7.3 already defines the

regression review), safety refusals, groundedness against her material, slot extraction accuracy, and cost per turn. Lelañea judges the voice; the team judges the rest. Nothing is promoted on the strength of a vendor announcement.

Prompt versions are pinned alongside model versions, and every turn records which pair produced it, so a regression can be traced to its cause rather than guessed at.

Staged rollout. Canary on a fraction of traffic, watch safety flags and cost, then promote, with an immediate rollback path.

Portability. Prompts and evaluations stay provider-neutral, because the assumption that one vendor will remain the right choice for three years has not held for anyone yet.

8.3 Support

Three channels already exist and none of them is support: the inbox is explicitly unmonitored (§3.17), the direct line to Lelañea is explicitly not a support channel (§3.18), and the agent is a coach. Someone who cannot log in needs a fourth door.

Help pages, authored as content and editable without a deploy, in the same voice as everything else: getting started, what the app is and is not, your data and how to export or delete it, troubleshooting, and how to reach a person.

A monitored support route, with an honest response time, clearly distinct from both the inbox and the direct line. Routing should be obvious: a product problem goes to support, something about the work goes to the app or to her, and anything in crisis goes down the crisis path.

A support queue in the admin area, showing account and technical facts by default and no conversation content. Reading a user's conversation to resolve a ticket requires their consent in the moment and is logged under the §7.2 rules like any other access.

Canned responses and macros as editable content, so that a recurring confusion becomes a fix in the help pages rather than thirty individual replies.

8.4 Accounts, age, and transactional email

The Terms require eighteen, so signup carries an affirmative age confirmation recorded with a timestamp alongside terms acceptance. If a user indicates during use that they are under eighteen, the app stops and points elsewhere rather than continuing quietly.

Email verification before module content. The address is the account's recovery path and, later, the binding for the inbox channels, so it needs to be real before anything personal accumulates behind it.

Transactional email is a distinct class from the announcements of §7.1: verification, password reset, email change confirmation, export ready, account deletion confirmation. These are service messages, carry no opt-out, and must never be used to carry marketing. Sender identity, deliverability (SPF, DKIM, DMARC), and a template set in her visual language all need setting up once, early.

Account recovery is a security surface. Given what accumulates in an account here, a recovery flow that is too helpful is a data-disclosure route.

8.5 Security and incident response

- **Encryption in transit and at rest**, with field-level encryption worth considering for journey, situation, and conversation content specifically.

- **Least privilege and role separation** (§7.2), multi-factor authentication for any role that can reach personal data, and access logging that the user can see.
- **Secrets stay server-side.** No provider keys in any client, and the API is the only path to a model.
- **Backups encrypted, restores tested**, with a stated recovery point and recovery time objective, and deletion propagating into backups within a stated window.
- **Dependency and supply-chain hygiene**, plus an external security review before launch, which is proportionate for special-category data.
- **A rehearsed incident response**, not an improvised one: named owner, severity levels, containment steps, and the GDPR 72-hour notification clock for personal-data breaches. The runbook should exist before it is needed.

8.6 Misuse and prompt injection

All user-supplied text is untrusted input, including chat, uploads, shared content, and anything arriving through the inbox by email or WhatsApp. Instructions found inside that content are data, never commands. This matters more here than in most products, because the inbox is an unauthenticated channel that reaches an agent capable of writing to a profile and instructing modules.

- **Agents hold least privilege.** Capabilities are explicit grants. A conversational agent cannot delete data, message Lelañea, or spend past a ceiling because it was talked into it.
- **Capability calls are validated server-side** against the calling user's own scope, so cross-user access is impossible by construction rather than by prompt discipline.
- **Provenance is carried into the prompt.** Her material is trusted; user content is not; retrieved passages are labeled by origin so the model can tell the difference.
- **The Supervisor agent enforces** staying in role, refusing clinical and therapeutic requests, refusing to disclose system prompts or fingerprint internals, and flagging manipulation attempts for review.
- **Rate limits and cost ceilings double as abuse controls**, particularly on the unauthenticated waitlist conversation of §3.1.
- **Attempts are logged and sampled**, and repeated abusive use has a defined response rather than an ad-hoc one.

8.7 Analytics and consent

The position: collect what improves the product and nothing else. Operational telemetry (errors, latency, token metering) is separable from product analytics, and only the latter needs consent.

- **EU users get a genuine choice** before any non-essential tracking, with refusal as easy as acceptance.
- **First-party and EU-hosted analytics** are preferred, and third-party advertising technology has no place here at all.
- **Conversation content never enters an analytics event.** Aggregate only, with the cohort minimums of §7.2.
- **The consent prompt is often the first thing a visitor sees**, so it should be quiet, plain, and in the register of §6.10 rather than a legal wall.

9. Feature set

Grouped so that phases can be cut from them.

Public Marketing site built from the about-surfaced documents, including an honest statement of why the app is being built and a video of Lelañea introducing it. Waitlist capture in two configurable modes, form or conversational, writing the same fields either way and carrying provenance into the profile on signup. Rate limiting and a cost ceiling on the conversational mode. Links to her YouTube, Spotify, and site.

Account and legal Sign-up and sign-in. Disclaimer and terms acknowledgement, recorded and gating. Crisis-language detection with an appropriate redirect. Data export and deletion.

Onboarding Personalized welcome. The four reads. The thirty discovery questions, resumable, producing the discovery baseline.

The journey All 17 modules registered and visible. Tier structure as a recommended spine with open navigation and first-class jumping. A user-controlled panel or drawer showing breadth and depth of coverage rather than completion. Invitations to deepen or explore untouched ground. Highlighted paths with visible reasons.

Conversation Two-panel layout with chat at the center and the working surface beside it. A multi-row input with send and microphone. Thinking indicator and streamed, typed-out replies. Per-turn timestamp with a plain-English disclosure drawer covering slots written, modules instructed, capabilities called, and tokens spent. Natural-language commands into modules, moods, events, and situations, with hand-off to specialist agents. A chat companion inside each module and ad-hoc conversation outside them. The login recap. Guiding and teaching registers. Knowledge-grounded responses in her voice.

Voice preferences User-set leanings across register dimensions. Adjustable mid-conversation by asking. Bounded so the essential fingerprint is never lost. Admin-defined limits on what may be adjusted.

Resources A drawer of films and reading drawn from her material and her influences, surfaced by the Curator agent and browsable directly.

Situations Conversational discovery of live circumstances rather than a form. A structured situation graph covering actors by role, setting, forces, tensions, emotions, responses, attempts, and open questions. Anticipated futures alongside actual events. An interactive situation map echoed back for confirmation, correction, or deletion. Consent before exploring. Lifecycle from emerging to closed.

Guidance Deterministic rules bounding what may be offered. Semantic evaluation of what would serve, weighted toward the recent. Reasons carried as data and answerable on request. Declines respected and remembered. An advisory council of perspective agents, quoted before it runs and delivered in her voice.

Budget Running dollar total for ordinary use. Cost estimates before expensive actions. User-set budgets for a period, later. Ceilings that explain rather than cut off.

The Values module Four lessons. The 265-value library with color-coding. Three reflection sets. Top-ten and top-five distillation. Alignment audit. Closing meditation with a timer. Value explorations on demand. A saved, revisitable values profile.

Module surfaces A per-module interactive artifact alongside the chat. A shared contract for rendering, slot access, progress, and exposing on-screen context to the companion. A growing vocabulary of reusable interaction patterns. Accessible by label as well as by color.

Practices A library across physical, mental, and emotional families. Guided delivery with timers. Practices attached to module moments and offered by agents in conversation. Post-practice check-in. Gentle history, never scored. Contraindication notes and alternatives.

The journey record Session synopses the user approves, edits, or regenerates. Automatic capture of module reflections. The user's own entries in the same stream. Action, insight, and tension counts per session. Newest first with a signpost for what is next. Search, filter, and export.

Profile Slot capture from conversation. A view where the user sees and edits what the app believes. Version history. Confidence and provenance. Contradiction handling.

Inbox Inbound email and WhatsApp capture bound to a verified account. Session-time triage: work through, write into the journey, route to a module, or clear. Crisis detection with automated response. A clear statement that the channel is unmonitored.

Contact with Lelañea Share-with-Lelañea from any surface, with context and a note. Her review queue. Asynchronous messaging and 1:1 session booking, behind a capability check ready to become tier-gated later.

Data and privacy Full visibility of everything held, with provenance. Chat history archive, view, and granular deletion. Deletion that propagates into derived data and embeddings. Export. GDPR rights end to end. Special-category consent.

Localization Location capture. Language preference. Externalized strings and locale-tagged content. Regionalized crisis resources and legal text.

Voice Corpus upload with per-document designation, sensitivity, and licensing. Evidence-backed pattern extraction. Admin review with weighting and contextual scoping. Categorized voice profiles and context-selection rules. Layered prompt assembly with retrieved exemplars. Versioning, side-by-side regression review, and drift monitoring.

Lelañea's console Shared-item queue with consent context. Direct messaging with a consent-scoped user panel, behind a capability check. Targeted, scheduled announcements with frequency caps and per-channel opt-out. Feedback themes.

Admin dashboard Growth and retention. Journey and phase-level drop-off. Aggregate content analytics with cohort minimums. Token usage and derived cost per user, session, module, agent, and model, with alerts, ceilings, and enough granularity to price on later. Safety flags and justified, logged transcript review. Role-based access with a full audit trail.

Reliability Retry without loss. Read-only degradation when generation is unavailable. Idempotent turns. First-token and total-turn deadlines. Disclosed fallback models. A model-independent crisis path. Status surface. Tested backups.

Model management An admin model registry: per-agent model and pinned version, fallback, parameters, and observed cost per turn. Prompt versions pinned alongside. Golden-conversation evaluation before promotion. Canary rollout and rollback. Turn-level record of which model and prompt produced it.

Support Help pages authored as editable content. A monitored support route distinct from the inbox and the direct line. Admin support queue showing account facts by default, with consented and logged escalation to content. Canned responses as content.

Accounts and email Age confirmation recorded at signup. Email verification before module content. Transactional email as a separate class from announcements, with sender authentication and branded templates. A recovery flow designed as a security surface.

Security Encryption in transit and at rest. MFA on privileged roles. Server-side secrets. Tested restores with stated objectives. External review before launch. A rehearsed incident runbook with breach-notification timing.

Misuse controls Untrusted-input handling for all user content including inbound channels. Least-privilege agent capabilities. Server-side validation of capability calls against the caller's scope. Provenance labeling in prompts. Supervisor enforcement. Rate limits and ceilings as abuse controls.

Analytics and consent Operational telemetry separated from product analytics. EU consent before non-essential tracking. First-party, EU-hosted preference. No conversation content in analytics events.

Operations Agent configuration and supervision. Slot-definition authoring, upload, and versioning. Knowledge base ingestion and tagging. Content updates without redeloys.

Design system Tokens, components, and assets from the supplied starter kit (\$6). Light and dark from the first screen, switching on system preference or explicit user choice. Desktop extensions built on the same tokens. Accessible state indication that does not rest on color alone.

Platform A versioned API covering every capability above. Token-based auth. Server-authoritative state and cross-device continuity. A streaming transport for chat. Client-agnostic content delivery from the JSON sources.

10. Release scope

Release 1: the app without its interiors

Release 1 ships no module detail at all, including Values. The modules exist as a navigable map with real names and real structure, and their interiors are empty. What release 1 proves is the thing every module will later sit inside: the shell, the themes, the conversation, the voice, onboarding, the journey record, and the machinery underneath.

In: the public marketing site and waitlist; accounts, legal gating, and data rights; the four-column shell in both themes; all 17 modules registered as a navigable skeleton; onboarding end to end including the discovery questions; and the conversation itself, wired to an agent speaking in her voice, grounded in the foundational documents, capturing slots, and metered per user. Desktop web, on an API built for clients that do not exist yet.

Out: every module interior, Values included. Also the inbox channels, direct messaging with Lelañea, practices, the advisory council, and the native apps.

The reasoning is that module content is the part most likely to change once real users are in front of it, and the part that depends most on Lelañea's authoring time. Everything underneath it does not. Shipping the container first means the first module arrives into a product that already works, rather than being built alongside the product that has to carry it.

It also makes release 1 honest about what it is: somewhere to talk to her, be understood, and see the map of what is coming.

Release 2: the Values module

Values arrives as the first interior, in two steps: the content and its visual surfaces, then the agent wired into them. It is the only module with finished content, and it exercises every primitive the others will need, which is lessons, a large library, a multi-state selection exercise, branching reflection, distillation, a saved artifact, and deep-dive knowledge retrieval. Once it works, the rest is repetition with new content, at whatever rate the authoring pipeline of \$4 can sustain.

Standing constraints on both

No payments, no plans, no tiers in release 1. The product is free to whoever is let in, and instead of charging it measures token usage per user from the first agent call, priced and displayed in **US dollars**. That is both the cost control the early product needs and the evidence the pricing decision will need later.

Data rights ship with accounts, whatever else slips. If a European user asks for their data or its deletion on day one, the answer cannot be that the feature is scheduled.

11. Implementation phasing

Proposed. This is a suggested sequence for the implementation plan rather than something the source material specifies.

Phase	Delivers	Done when
0. Foundation and shell	Leaf fork off a tagged Daybreak release; app scaffold; design tokens and components installed with light and dark modes and a theme toggle; the four-column shell and the public layout standing up against the prototype; content files loaded and validated against their schemas; the API boundary, storage split, and auth model settled	The shell renders in both themes and matches the prototype's behavior for collapsing panes and drawers; content is queryable through the API rather than only in-process; CI green
1. Public face	Marketing site from the about documents; her intro video; waitlist in both modes behind one config switch; help pages and a support route; outbound links	A stranger can find her, understand the work, and join the list in either mode without a code change
2. Gateway	Accounts; age confirmation; email verification and the transactional email set; disclaimer and terms gating; crisis redirect; analytics consent; location capture; baseline export and delete; roles and audit logging	Nobody reaches the app without acknowledging both, and a user can leave with their data
3. Skeleton	17 modules registered; the map with open navigation; the coverage panel; the module-surface contract	The whole journey is visible, a user can jump anywhere, and coverage is shown without a completion score
4. Onboarding	Welcome, the four reads, the 30 questions, discovery baseline slots	A new user completes onboarding and the app can quote them back
5. The conversation	The chat experience with streaming, thinking state, and live slot formation beside it; voice fingerprint v1 in the prompt path ; grounding in the foundational documents; slot capture from conversation; per-user token metering; the model registry and degraded-mode behavior	A user can talk to her about anything, be answered in her voice from her material, and see what each turn recorded and what it cost
6. The companion	Login recap; guiding and teaching registers; per-turn disclosure drawer; voice preference leanings; the journey record with session synopses	Returning feels like a session resuming, and a user can read back what was said and correct it
7. Values, content-first	All lessons, library, exercises, reflections, meditation, values profile, each with its visual surface	A user completes Values by working with it on screen rather than only reading, and their profile persists
8. Values, agent-wired	Module chat companion; knowledge grounding in the module content; explorations; module slot capture	The agent references the user's own answers and her actual material inside the module
9. Situations	Conversational discovery; the situation graph; the interactive map and synopsis confirmation; consent gating	The app can show a user the shape of what they are living through, and be told it has it wrong

Phase	Delivers	Done when
10. Guidance	Deterministic rules; semantic evaluation of what would serve; recency weighting; reasons as data; the advisory council in her voice	Highlighted paths change as the user changes, and every one can be questioned
11. Budget	Running dollar total; estimates before expensive actions; ceilings that explain	A user always knows what they have used and is never surprised by a spend
12. Practices	Practice library across the three families; guided delivery and timers; agent- and module-triggered offers; gentle history	A user is offered the right practice at the right moment and can find it again
13. Inbox	Verified email and WhatsApp capture; session-time triage; crisis auto-response	A user sends a thought on Tuesday and works through it on Sunday
14. Contact with Lelañea	Share-with-Lelañea from any surface; her review queue; messaging and session booking behind a capability check	A user reaches the real person, with context, and knows what to expect back
15. Transparency	Profile view; chat archive and granular deletion; version history; confidence and provenance display	The user can see, correct, and remove everything the app holds
16. Voice workshop	Designated upload, evidence-backed extraction, review with weighting and scoping, contextual rules, versioned regression review, drift watch	A new podcast season improves the app's voice without an engineer
17. Admin dashboard	Growth, journey and phase drop-off, aggregate content analytics, cost dashboards, safety flags, roles and audit views, the model registry and evaluation history, the support queue	Decisions about what to build next come from data rather than instinct
18. Lelañea's console	Shared queue, messaging with consent-scoped context, targeted announcements, feedback themes	She can reach her users, and read what they chose to send her, without leaving the app

Release 1 is phases 0 through 6. Release 2 is phases 7 and 8. Everything from phase 9 onward is sequenced but not committed, and much of it could be reordered by what release 1 teaches.

Theming and the layout shell come first, ahead of feature work, so there is something visual to build against from the outset. That is why phase 0 carries the shell as well as the scaffold: every phase after it fills panes that already exist rather than inventing layout alongside behavior.

Security, misuse controls, and degraded-mode behavior are not phases. They are conditions on the phases that introduce the surfaces they protect: untrusted-input handling arrives with the first agent in phase 5, capability scoping with the first capability, and the incident runbook before any real user reaches phase 2.

Phases 7 and 8 are deliberately split. Content correctness is verifiable without agent behavior in the way, and agent quality is much easier to judge against content that is already right.

Three things sit outside the table's order. Data rights start in phase 2 in baseline form, export and delete, because they are a legal floor rather than a feature, with phase 15 deepening them into granular, provenance-aware control. Role-based access and the audit trail also belong in phase 2, because an audit log that begins halfway through is not an audit log. And the storage decision in §5 lands in phase 0, since relational and vector stores are hard to separate later and every phase after it writes into whatever is chosen.

Token metering starts with the first agent call in phase 5. Retrofitting attribution across sessions and agents is far harder than emitting it from the start, and the dashboard in phase 17 is then only a view over data already collected. With no revenue in release 1, this is the one operational number that matters from day one. What phases 17 and 18 add is the surfaces, not the instrumentation beneath them.

The voice fingerprint splits the same way. A first version has to exist before phase 5, because an agent that speaks in a generic voice from its first conversation teaches users what the app sounds like, and that impression is expensive to undo. That first version can be assembled semi-manually from her existing corpus. Phase 16 is the workshop, the tooling that lets it be maintained, weighted, scoped, and improved without an engineer in the loop, and it should land before her corpus grows much, since the value compounds with every episode she records.

The journey record moves into release 1, at phase 6, because without module work there is less to record and it matters more: it becomes the main evidence that the app is paying attention. Its module-reflection stream fills up later, from phase 7.

Situations land just after Values deliberately: knowing what someone is actually living through is the strongest single input into what would serve them, and a recommender without it reasons about a person in the abstract. Recommending what would serve also needs something to reason over, which means a completed onboarding, some real sessions, and ideally a finished module. A recommender running on a thin profile produces confident nonsense. The map therefore ships in phase 3 with open navigation and no highlighting, and the highlighting arrives in phase 10 once it can be right. Budget follows immediately, because phase 10 is the first thing expensive enough that a user should be asked before it runs.

Practices are held back to phase 12 for a content reason rather than a technical one: only one is written, and the library needs authoring before the surface is worth building. The closing meditation ships inside Values in phase 7 regardless. Both the journey record and practices are strong candidates to pull forward if mobile moves up the roadmap, since a timed practice with audio and a journey entry written on a phone are the two most native-feeling things in the product.

Every later module carries its own design work, not only its own content. Expect a short discovery step ahead of each, asking what this module's artifact is and whether it can be composed from patterns that already exist, and expect the first attempt at some of them to be wrong. Values is where the vocabulary starts and where the cost of getting a surface wrong is lowest.

Localization is not a phase but a constraint on every phase: externalized strings and locale-tagged content from the first line, so that adding a language is later work rather than retrofitted work.

Monetization is a phase beyond this table, covering plans, checkout, entitlement, and the tier gating that phases 14 and 18 are built ready for. It should be specified once there are real usage figures to specify it against, which is what phase 5 starts collecting.

Native iOS and Android also sit after this sequence rather than inside it. The point of settling the API boundary in phase 0 is that mobile should then be a client build rather than a second

implementation, and a useful check on every phase before it is whether a native client could have consumed what was just built.

12. Guardrails

- **Never therapy.** No diagnosis, no treatment, no clinical language. The agent declines and redirects rather than improvising.
- **Crisis handling is a designed path** rather than a caught exception, it applies before signup as well as after, and it does not depend on a model being reachable.
- **Failure is spoken plainly.** Nothing the user typed is lost, provider errors are never shown raw, and a substituted model is always disclosed.
- **Instructions inside user content are data, never commands.**
- **The waitlist conversation is a taste, not a session.** It gathers what a form would gather and shows what the app sounds like. It does not coach someone who has not yet agreed to anything.
- **Grounded rather than improvised.** The agent answers from her material. Where it has nothing, it says so.
- **It sounds like her; it never claims to be her.** The voice is hers by design and disclosed as AI without ambiguity. Where a user wants the actual person, that is \$3.18, and the app should say so rather than let the resemblance answer for it.
- **Preferences filter, they never override.** No combination of leanings produces a generic assistant, removes her ethics, or softens a refusal.
- **Client material is not raw material.** Nothing from a 1:1 session enters the fingerprint without informed consent and de-identification. Coaching confidentiality outranks product quality every time.
- **Inference is visible.** Every slot carries how it was derived and how confident the app is. Nothing about a user is asserted from a black box.
- **Nothing is understood invisibly.** Every turn discloses what it changed, in plain English, one click away. The app does not quietly revise its picture of someone while talking to them.
- **The user owns the picture.** They can see, correct, and remove what the app holds.
- **Deletion is real.** What a user removes goes from the derived picture and the embeddings, not only the transcript.
- **The journey record is the user's rather than the app's.** Nothing is written to it without approval, and anything in it can be edited or removed.
- **Situations are held, not handled.** Nothing about someone's life is recorded as fact without them confirming it, the app asks before opening a painful subject, and no is a complete answer. Anything can be corrected, muted, closed, or deleted.
- **Naming a fear, not rehearsing it.** Anticipated futures are tracked to loosen their grip. If returning to one is feeding rumination rather than easing it, the app should notice and let it be rather than keep offering it.
- **Other people are not subjects.** The app holds the roles it needs to understand a user's situation and does not accumulate a picture of people who never consented to being in it.
- **Development is a tuning signal, never a grade.** The app benefits from sensing where someone is in examining their own conditioning, because it changes pace and register. It must never rank, score, or display that as a level. The Hawkins scale sits inside the content as a

teaching frame, and the distance from there to a number on a user's profile is short and must not be crossed.

- **Coverage is never a score.** Breadth and depth describe where attention has gone. They are not maximized, ranked, or compared.
- **Silence is allowed.** Nothing pressures completion. No streaks, no scores, no nudges dressed as encouragement.
- **Recommendation serves the user, never engagement.** Nothing is surfaced because it keeps someone in the app. If the honest answer is to close it and go for a walk, the app should be able to say so.
- **Every suggestion can be interrogated.** Ask why, get the real reason from the real data. A decline is respected and remembered.
- **Perspectives, not verdicts.** The advisory lenses inform a choice the user makes; they never make it. Attribution to a named thinker stays honest about what that thinker actually held.
- **Practices carry their own care.** Contraindications are stated, alternatives offered, and no somatic or breath practice is pressed on someone who is distressed.
- **No spend without consent.** Anything materially expensive is quoted first. Ceilings explain and offer a choice rather than cutting a conversation off.
- **Features are explained, never sold.** What something is for, what it costs, and why it might help, in the user's interest, in plain terms, with no urgency, scarcity, or manufactured need.
- **Nothing intimate leaves without a decision.** Sharing with Lelañea is explicit, previewable, and revocable. The inbox says plainly that it is unwatched.
- **Staff see less than they could.** Lelañea's view of a user is scoped by what that user shared. Transcript access by anyone running the app is exceptional, justified, logged, and visible to the user.
- **Analytics are aggregate.** No view of what people value can be narrowed back to a person, and cohort minimums apply before a figure is shown.
- **Announcements respect the character of the app.** Frequency caps, honored opt-outs, and a bias toward being found rather than pushing.

13. Success measures

Proposed.

Users return unprompted. Onboarding is finished rather than abandoned, even across several sittings. Values is completed by most who start it. Users correct their profile, which is evidence they trust it enough to bother. Sessions get longer and more specific over time. And the measure that matters most: the recap is accurate often enough that users stop being surprised by it.

14. Open decisions

Carried from the content's own review notes, the grounding brief, and the sections above.

Legal, blocking before launch Terms of Use has an unfilled effective date and support email. It references a Privacy Policy that does not yet exist. The governing jurisdiction clause names no jurisdiction. Crisis guidance says "contact local emergency services" without naming a resource, so decide whether to localize or set a named default.

Content The disclaimer contains its own creator bio that differs from the standalone one, so pick a canonical source. Thirty discovery questions before any module content is a heavy gate, so consider a required core plus an optional deeper set. `{{first_name}}` needs a fallback. 249 values have no exploration, so define the fallback experience.

Data and architecture Settle the storage split in §5. Relational as source of truth with a parallel vector index is the proposal here, and it needs confirming before phase 5 writes anything. Then: retention periods; whether deletion is immediate or soft with a window; data residency for European users; the processor terms and disclosure position for sending journey content to a model provider; and the lawful basis for special-category data.

Slot taxonomy Who authors it, how it is versioned, and how a definition changes without orphaning the values already captured against it. Also how far it goes: the useful taxonomy is larger than any first pass, so the mechanism for growing it matters more than the initial list.

Situations How much structure is right before it stops fitting real lives. The ontology in §5 is a first attempt and should be expected to change once real conversations are visible. Then: what closes a situation and who decides; whether the app may raise one unprompted or only when the user opens the door; how a situation is visualized, which is the hardest of the module surfaces because the subject matter is a person's actual difficulty; how situations interact with special-category data and retention, since this is the most sensitive material the app holds; and where the line sits between tracking an anticipated fear usefully and becoming a worry log that entrenches it.

Guidance and the council Where the line falls between deterministic rules and semantic judgment, and how recency is weighted. Which perspectives the council holds, who writes each lens, and how fidelity is checked so a Jungian reading is actually Jungian. This needs Lelañea's sign-off, since a caricature in her app is her reputation, and it needs a rule for how far a paraphrased "Jung might say" can go before it becomes a claim nobody can support. Whether the council is user-invoked, agent-suggested, or both. And how a recommendation is evaluated after the fact, because if the app cannot tell good suggestions from bad ones it cannot improve them.

The conversational surface Which preference dimensions ship, whether they are toggles or sliders, and which are locked to her. Whether the disclosure drawer is collapsed by default, probably yes, though the cost line inside it is the awkward one: showing what a turn cost is honest, and also asks someone to watch a meter while doing emotional work. The suggestion here is that the running total belongs in the budget view and the per-turn figure stays behind the chevron, but it is worth deciding deliberately. Also: what the microphone does with audio and where speech-to-text happens, since voice notes about a marriage are not ordinary telemetry; and how the right-hand panel behaves when the conversation is doing something the panel cannot show.

The map and coverage panel The form is open: tabs, a map, a drawer, or a combination. Harder: how breadth and depth are computed from slots, time, returns, and volume without a number creeping back in through the side door, and how an invitation to deepen stays an invitation rather than becoming a nag. Also whether the panel defaults to open or closed for a new user, which quietly signals what the app thinks it is for.

Module surfaces Mostly unanswerable now, by design, but three things are worth settling early: how narrow the framework-to-module contract should be, since it constrains every module that follows; whether to commit to a shared pattern vocabulary or let each module design freely, given the native-rework cost of the latter; and who does this design work, since it is the one part of the product that cannot be derived from Lelañea's written material and will need drawing rather than transcribing.

The voice fingerprint The blocking question is **client transcripts**. They are the richest source of how she actually coaches and also the most constrained: ICF confidentiality, third-party personal data under GDPR, and clients who consented to a session rather than to a training corpus. Decide whether they are used at all, and if so under what consent, what de-identification, and whether they inform patterns without ever being retrievable as exemplars.

Then: where the line falls between voice and content when a phrase is both; how fidelity is judged and how often she reviews; how much fingerprint fits in a prompt budget, since it is paid for on every call and therefore lands directly in §3.20; and the IP question, which matters beyond this app, since the fingerprint is derived from her work and is arguably hers, and that needs settling before the framework offers the same arrangement to a second expert.

Practices and the journey record A practice library needs authoring, since only the closing meditation exists today. Decide the taxonomy within the three families, who writes the contraindication notes, and whether practices live as their own module, a cross-cutting library, or nodes on the facilitation map. For the journey record, decide whether synopsis approval doubles as profile confirmation (§3.16), what happens to inferred slots when a user edits a synopsis rather than approving it, and whether agents may read back over past entries or only over the profile derived from them. That last one is a privacy question as much as a design one.

Inbox and contact with Lelañea How an email address or phone number is bound to an account, and what happens to messages from unbound senders. Whether WhatsApp is Business API or something lighter at first. Lelañea's own capacity and stated response times, which are sharper without tiers, since nothing limits who can write to her in release 1 and her attention is the scarce resource. Whether sharing is open to everyone from the start while messaging waits. Whether a direct exchange with her needs its own agreement separate from the app's terms.

Operator surfaces The consequential one is what Lelañea sees by default about a user she is messaging. The proposal here is consent-scoped only (§7.1), and it needs her agreement because it is narrower than she might expect. Then: which announcement channels at launch and what frequency cap; the cohort minimum before an analytics figure is shown; per-user cost ceilings and what happens when one is reached; how long analytics and audit logs are retained; who besides Lelañea holds an admin role; and whether transcript sampling for quality needs its own consent at signup rather than resting on the terms.

Budget The unit is settled: US dollars (§3.20). What remains: the ceiling per user and per period in release 1; what happens at zero; whether estimates are shown as ranges; and how wrong an estimate can be before it stops being honest.

Commercials, later Deferred, but worth naming now so v1 collects what the decision will need: what the tiers are and what each buys; whether pricing is per month, per module, or something else; whether there is a free tier and what it includes; and what per-user ceilings apply while the product is free, given there is no revenue absorbing a heavy user. These should be answered from phase 6 data rather than from comparable apps.

Localization Which languages, on what trigger, and by whom. Machine translation would flatten her voice, so this may mean human translators or a deliberately narrower experience in other languages.

Platform Where the API boundary sits for a Daybreak leaf app needs settling in phase 0 (§5). Native apps also raise three product questions worth answering before the mobile phase rather than during it: whether app-store review of a coaching app forces changes to how the disclaimer gate is presented; whether push notifications are wanted at all, given that a check-in nudge sits

awkwardly against the guardrail that nothing pressures completion; and whether the long-form reading, the four onboarding documents and the value explorations, should be available offline.

Waitlist Which mode launches, and whether both run at once as a comparison. What the conversational mode does when someone tries to turn it into a session. Whether an email address is required to end the conversation or optional. How long waitlist answers are retained for people who never sign up, and what the lawful basis for holding them is.

Prototype conflicts The five disagreements in §6.12 need settling: whether the public site publishes tiers before anything can be bought, which of the three undesigned features get surfaces, and whether the red mark keeps the name “let go of”. Two of the five are now settled: the budget unit is dollars, and “module closed” does not carry over.

Design system The secondary-text token change proposed in §6.2 needs accepting or rejecting, since it touches every meta line in the product. The three fonts and the icon family are substitutions the kit itself flags (§6.11), so confirm or replace them. The supplied wordmark reads “Lelanea” without the tilde and needs redrawing. Decide whether the theme defaults to system preference or to light, and whether a user’s choice is a slot like any other preference. Then the extension question: who designs the desktop patterns the kit does not cover, and how the design system is versioned once it starts changing, since it will be edited alongside the product rather than frozen.

Value state colors The four value states in `values_module.json` currently carry generic hex values chosen before this palette existed. The palette has close semantic equivalents already: sage forest for confirmed, dusted ochre for wavering, terracotta red for let go of, and heather amethyst for future self. Worth remapping, provided the state labels stay as written, since the source worksheets name the colors explicitly.

Reliability and models The first-token and total-turn deadlines, and what the app says when they pass. Whether a fallback model answers at all or the app simply waits and admits it. Which models are assigned to which agents at launch, and the recovery objectives for backup and restore. Who owns the evaluation suite, and what the standing golden conversations actually are, since a suite nobody maintains is worse than none.

Support and operations Who staffs support and at what stated response time, given that this is currently an unowned role. Whether help pages live in the same content pipeline as the modules. The analytics tool, its hosting region, and whether operational telemetry is genuinely separable in whatever is chosen.

Content authoring The workstream in §4: who drafts, how she reviews, how long a module takes, and whether the Values shape generalizes. With release 1 carrying no module content at all, this question moves from important to urgent, because release 2 is entirely gated on it.

Design Guiding versus teaching needs a decision on what triggers the switch, whether the user can ask for one register, and how it is bounded. “Integration” is currently both a Values phase-tier and an app tier name, so rename one.

Naming The framework overview calls the platform **Sunrise**; the grounding brief works against **Daybreak** as the framework layer with leaf forks. This document uses Daybreak, but the two should be reconciled so the build prompt uses one vocabulary.

Structure The framework overview lists a different module set (Core Values, Future Self, Lotus of Life, Standards, Boundaries, Liminality, Check-Ins) than the 16-module journey in `lelanea_module_structure.json`. That document marks its list illustrative and the JSON is

authoritative, but Future Self, Liminality, and Check-Ins do not appear in the journey at all and may be intended.

Waiting on Lelañea Collected here because the build cannot proceed past certain points without them.

- A video of her introducing the app, for the public site (§3.1).
- Confirmation or replacement of the substituted fonts and icon set, and a wordmark with the tilde.
- The Terms effective date, support email, and a Privacy Policy.
- Corrected Module 01 chart artwork, which prints old numerals in rows 2 and 4, out of sync with the sequence in the JSON.
- Sign-off on the voice fingerprint, and on each advisory lens.
- A decision on whether client transcripts may be used for voice work, and on what basis.
- The practice library beyond the closing meditation.
- Her stated response times for anything users send her directly.

Appendix A: the document set

Everything below is delivered with this document. Paths are as they appear in the accompanying archive.

The product description

File	What it is
lelanea-product-description.md	This document, in Markdown. The editable source.
lelanea-product-description.pdf	This document, typeset for reading and circulation.

Content ([content/](#))

Authored content, transcribed from Lelañea's own material and corrected only for typography. Described in §4.

File	What it holds	Referenced in
<code>lelanea_foundational_documents.json</code>	Welcome, philosophy, mission, creator, lineage, disclaimer, terms, each tagged with the app surface it belongs to	§3.9, §1
<code>lelanea_module_structure.json</code>	The 17-module journey: onboarding plus 16 modules across 5 tiers, with Module 01 built out in full	§3.5, §3.10
<code>onboarding_discovery_questions.json</code>	30 discovery questions, two of them branching on a yes or no	§3.9
<code>values_module.json</code>	The Values module: 4 value states, the 265-value library, 10 authored steps	§3.10, §6.12
<code>values_reference_framework.json</code>	The 22-point editorial brief the value explorations are written against	§4
<code>value_explorations.json</code>	Long-form explorations, one per value, on a common 25-section schema. 16 of 265 written	§3.10

Design ([design/](#))

File	What it is	Referenced in
<code>lelanea.html</code>	The approved prototype. The reference for layout, theming, and behavior	§6.1, §6.12, §3.3
<code>lelanea-design-prompt.md</code>	How the prototype should be read, and what in it is real versus placeholder	§6.1
<code>LeLanea_Design_System.zip</code>	Tokens, seven components, the mobile UI kit, brand assets, and a <code>SKILL.md</code> usable directly in Claude Code	§6

Reference ([reference/](#))

Inputs rather than deliverables. Kept because the product description condenses them and someone will eventually want the originals.

File	What it is	Referenced in
<code>lelanea-app-spec.md</code>	The technical grounding brief: Daybreak primitives, the leaf-fork boundary, slot scopes	§5
<code>LeLanea_Framework_Overview.pdf</code>	The non-technical account of the five concepts, the slot taxonomy, and the agent family	§5

Precedence

Where these disagree, the order is: **this document** for what the product is, what it says, and its vocabulary; **the prototype** for how it looks and behaves; **the design system** for tokens and components; **the content files** for anything authored by Lelañea, which is never paraphrased in the build. The reference documents are context rather than specification, and §14 lists the places they conflict with this document.

Appendix B: content inventory

17 modules across 5 tiers. 12 phases in Module 01. 10 authored steps. 265 values. 4 value states. 16 value explorations on a 25-section schema. 30 discovery questions. 7 foundational documents. A 22-point editorial framework. 1 authored practice of a library still to be written. A visual surface per module, discovered as each is built. A recommended spine, openly navigable. A voice fingerprint built from her corpus and maintained as a living asset. A situation ontology discovered through conversation. A journey record, practices, inbox, contact, and data rights layered on top. A starter design system with light and dark tokens, seven components, and the lotus. An approved prototype covering the public site and the four-column app shell. US English throughout. App name **Lelañea**, domain **lelanea.com**. Desktop web first, native iOS and Android to follow. Release 1 ships the container; the Values module follows in release 2.